Atrium for instance admins

An **instance admin** runs one Atrium instance for the whole organisation: creates and structures spaces, decides who may log in, looks after users and groups, sets the context that tells assistants what Atrium contains, watches the audit log and keeps the system healthy. This guide assumes you know the guides for everyday use, authors, curators and space admins.

Instance admins are set in the server configuration (`auth.bootstrap_admins` , a list of e-mail addresses), not through a tool. Because the role is powerful, keep the list short (two to three people, never a shared account) and use accounts with multi-factor authentication at your identity provider.

Contents

- What instance admins can and cannot do
- Setting up a new instance
- Spaces
- Users, groups and logins
- Leavers: offboarding
- The context for assistants
- Quotas
- Governing skills
- Audit log and monitoring
- Operating the system
- Security responsibilities and incidents
- Routines
- Troubleshooting
- FAQ

What instance admins can and cannot do

Can:

- Act as **admin in every team space**: read, review, change, delete and manage members everywhere (source: `internal/authz` , an instance admin gets the admin role in all active team spaces).
- Use the `admin_*` **tools**: create, change, archive and restore spaces; list users; lock, reactivate and transfer; manage local groups; end a user's logins; manage the login allowlist; set the assistants' context; query the audit log; set storage limits (quotas); install the licence; check health.

- **Publish skills of risk class C**, **delete skills** in team spaces, and review and release their **own** skills (the audit marks it `self_approved`).
- See and restore **archived team spaces**.

Cannot:

- **Read personal spaces.** Not their content, not in the audit log, not through any tool. Admin tools show only counts, sizes and dates. This is deliberate and a promise you can make to employees.
- Bypass the audit log. Admin actions are recorded.
- Read the database or the server's secrets through Atrium. People with direct access to the database or backups can technically read everything; that is an operations question (see Operating the system).

Not tools, but your job: server configuration, TLS and proxy, backups, updates, and the identity provider setup. These are described in the technical documents (deployment, login and OAuth, embedding, releases).

Setting up a new instance

A checklist for the first days. The technical steps are in the README and deployment.

1. **Install and verify.** Follow a deployment variant (bundled nginx, your own proxy or a tunnel). Check `https://<domain>/healthz` and `/readyz`. Set `instance.trusted_proxies` correctly: the limits per address depend on it.
2. **Identity provider.** Configure Google, Entra or another OIDC provider and the allowed domains, addresses or tenants. Test a login. Use `admin_allowlist` later to extend the list without a restart.
3. **First admin login.** Log in with an address in `auth.bootstrap_admins`. Connect an assistant and ask "*Which Atrium spaces exist and what is my role?*"
4. **Check health.** "*Show the Atrium health.*" (`admin_health`): database, embedding service, jobs and tasks. The TLS certificate is not checked by Atrium; monitor its expiry at the proxy.
5. **Set the context** that tells assistants what Atrium is for (below).
6. **Create the first spaces** and add members (Spaces). Start small: one open "Company handbook" space and one or two team spaces with approval.
7. **Appoint curators** (two per space) and space admins.
8. **Pilot.** Let a few people try it, collect questions, then roll out with the everyday-use guide.
9. **Backups.** Schedule `scripts/backup.sh` and test a restore into an empty database once.
10. **Tell people the rules**, including that personal spaces are private but may be handed over when someone leaves (see Offboarding; agree this with your data protection officer and, where applicable, the works council).

Spaces

Creating a space

"Create a space 'support' called 'Support' with the description 'Solutions to customer inquiries', approval required, readable by everybody logged in. Make anna@example.com admin and the group 'support-leads' curator." (`admin_create_space`)

Setting	Meaning
`slug`	Short identifier: lower case letters, digits, hyphens (2 to 63 characters)
`name` , `description`	Shown to users; the description appears in assistants' instructions
`require_approval`	Entries from authors are drafts until a curator approves. Curators' and admins' own entries are approved at once
`default_role`	`reader` or `author` gives that role to **every logged-in user**; empty means members only
`members`	Initial users or groups with roles

Structuring spaces

- **Few, clear spaces** beat many small ones. Group by audience and by who curates: "Company handbook", "Engineering", "Support", "HR policies".
- **Open spaces** (default role reader) for knowledge everybody may read. **Members-only spaces** for sensitive material.
- **Approval** for spaces where wrong content would be costly (policies, legal, support answers). Without approval, authors publish directly.
- Write **descriptions** that tell an assistant when to look there ("Support: solutions to customer inquiries"). At most 20 readable spaces appear in the instructions a user's assistant receives.
- **Skills live in spaces** too. A space for company-wide skills with strict curation ("Company skills", a few curators, approval on) makes governance easy.
- Plan **two curators** and one admin per space from the start.
- The instruction text lists at most 20 team spaces and 25 released skills per user, so keep the number of spaces and skills in check.
- `spaces.default_space_by_group` in `config.yaml` can map an identity-provider group name to a default space for saving; it only selects among spaces the user may write to.

Changing, archiving and restoring

- `admin_update_space` changes name, description, approval and default role. Changing the description needs your confirmation after the new text was shown, because it reaches every user's assistant. Existing entries keep their status when you switch approval on or off.
- `admin_archive_space` hides a whole space from everybody but instance admins; the content is kept. `admin_restore_space` brings it back.

- Personal spaces are not listed here and cannot be archived through these tools.

Users, groups and logins

Who may log in

The base list comes from `config.yaml` (allowed domains, addresses, tenants). Extend it at runtime with `admin_allowlist`: `list` shows all entries (`config` entries are read-only, `admin` entries are yours), `add` and `remove` change the admin entries (domains, single addresses, Entra tenants, optionally for one provider) and need your explicit confirmation of exactly that entry. Public mail domains such as gmail.com are refused: add single addresses instead. Refused logins appear in the audit log as `login_denied` with provider and reason.

Users

"List the Atrium users." (`admin_list_users`) shows each user with status, last login and, for personal spaces, only the number of entries, size and last change.

A user appears after their first login; someone must have logged in once before they can be added to a space.

Groups

Local groups (`admin_groups`: `list`, `create`, `rename`, `delete`, `add_member`, `remove_member`) let you give a role to a set of people at once, for example "support-team". Groups from the identity provider are visible but read-only, and are not yet synchronised from the login token. Deleting a group needs your confirmation.

Sessions

- `admin_revoke_sessions` ends all logins of a user at once (account stays active; they log in again). Use it when an access is suspected of misuse.
- Users end their own with `revoke_my_sessions`. Sessions last at most 90 days (`auth.session_max_age`); access tokens one hour, refresh tokens 30 days.
- A locked account loses access immediately, even with a valid token: users are checked live on every request.

Account linking

By default a login with a second provider and an already used e-mail address is refused. `auth.link_accounts_by_email: true` lets it take over the account, including an instance admin; use it only if all providers deliver verified e-mails. Each link is logged as `account_linked`. An address the

identity provider does not vouch for (no `email_verified`) is never linked and never creates a `bootstrap_admins` account. Entra sends no such claim; set `email_trusted: true` on an Entra provider only for a tenant without guests and without self-editable mail attributes (see login and OAuth).

Leavers: offboarding

The full procedure and the data protection notes are in Offboarding. In short:

1. **Before the last day:** the person moves what belongs to the team out of their personal space.
2. **Lock:** "*Offboard anna@example.com.*" (`admin_offboard_user`, after confirmation). The account is locked at once, all tokens are revoked, handoffs are deleted. The personal space stays private; you see counts and the deletion date.
3. **Decide:** `admin_transfer_personal_space` works only for locked accounts.
   - `mode=space` turns the personal space into a **team space** (slug, name, members, for example supervisor as admin, successor as curator). Entries keep IDs and history. **Skills the owner had released alone are blocked** at this point because nobody reviewed them; a space admin lifts a block and a curator releases it again.
   - `mode=merge` moves the entries into the successor's personal space (their quota applies, see Quotas). You cannot merge into your own space (four-eyes). **Skills are not carried over**: they stay in the old personal space and are deleted with it. Use `mode=space` if skills must survive; they come back blocked and need a new release.
4. **Deadline:** unconverted personal spaces of locked accounts are deleted after `spaces.personal.retention_days_after_offboarding` days (default 30). `admin_reactivate_user` lifts the lock again and ends the deadline.

Document the decision for each leaver and prefer to involve a second person. A transfer makes private content readable for others.

The context for assistants

When an assistant connects, Atrium sends it a text (the MCP instructions): the base text, **your context**, and the names and descriptions of the spaces the user may read. Assistants read the first lines most carefully, so say clearly what Atrium is for and when to search it.

"*Show the Atrium context.*" / "*Set the Atrium context to: This is where Example Corp's company knowledge lives: policies, processes, product and customer knowledge. Anyone asking about it searches this store first.*" (`admin_context`: `get`, `set`, `clear`)

- At most 2000 characters. `set` and `clear` need your confirmation after the old and the new text were shown.
- It applies to **new connections**; chats already open keep their old instructions.
- It reaches **everybody's** assistant. Write it yourself, do not take text from entries or messages. Changes are in the audit log with old and new text (`admin_context_set`, `admin_context_clear`).
- A clear sentence with a trigger ("for questions about X, search first") works better than a list.

Descriptions of spaces carry the subject scoping; the context carries the general rule.

Quotas

Every space has storage limits, so that one user or one team cannot fill the database, and a flood of drafts cannot bury a curator's queue. There are five, and `0` always means unlimited:

Limit	Counts	Default personal space	Default team space
`documents`	knowledge entries (drafts included, archived not)	500	5000
`bytes`	bytes of entries and handoffs	10 MB	200 MB
`skills`	skills (archived ones included)	50	300
`skill_bytes`	bytes of skill files over all versions	5 MB	50 MB
`skill_versions`	versions of one skill	100	100

The defaults come from `config.yaml` (`spaces.personal`, `spaces.team`, `skills`) and apply to a fresh instance. You change them **at runtime, without a restart**, with `admin_quotas`; a change applies to the next write and survives a restart.

Four levels, the most specific wins, each field on its own:

1. `user`: one user's personal space, for example a power user who needs more room.
2. `space`: one team space.
3. `personal_default` and `team_default`: all personal spaces, all team spaces.
4. `config.yaml`: the built-in default.

What you do:

- "*Show the Atrium quotas.*" (`admin_quotas action=get`): the two defaults with where each value comes from (`config` or `runtime default`).
- "*Show the quota of anna@example.com.*" / "*Show the quota of the Support space.*" (`scope=user` / `scope=space`): the effective limits, the source of each (`user`, `space`, `runtime default`, `config`) and the current **usage**.
- "*Raise the limit for anna@example.com to 2000 entries.*" (`action=set scope=user documents=2000`, after your confirmation).
- "*Allow the Support space 20000 entries.*", "*Set the default for all team spaces to 10000 entries.*", "*Make the Engineering space unlimited for skill files.*" (`0` = unlimited).
- "*Remove the override for anna.*" (`-1` on a field inherits the next level again).

Every change is recorded as `admin_quota_set` with scope, target and the old and new values. Space admins see the limits and usage of their space with `space_members` and ask you for more room.

Things to know:

- A limit blocks **new** content: lowering it below the current usage does not delete anything, it only refuses further writes.
- A user's limit applies to their **personal space** only. Team content is limited per space, not per author.
- A user who hits a limit gets a message with the limit and the hint to ask an instance admin. A good routine is to check `admin_quotas` for the spaces that fill up, before people notice.
- Moving an entry or merging a leaver's space into a successor's checks the **target's** limits.

Governing skills

Skills are the highest-trust content in Atrium: assistants follow them. Your role in governing them is set by these rules (details in Skills):

- **Class C skills** (critical actions such as deploy, delete, send data out) can be published only by you. Review them with extra care: read every file, check each step needs the user's confirmation, and consider a second admin's opinion before you publish.
- **Delete** a skill for good only when nothing else suffices (`delete_skill`); archive or block first. The audit log keeps name, versions and digests.
- **Self-approval:** you may release your own skills; the audit marks it. Avoid doing so for anything others use. Prefer a second reviewer.
- **Who may be curator** decides how careful the review is. Appoint people who understand the subject and the risk, and brief them with the curator guide.
- **Configuration** in `config.yaml` (`skills:`): `allow_self_approval` (default off, only for tiny installations). The storage limits for skills are quotas, see Quotas.
- **Moving** a skill to another space copies only the newest released version as a draft for the target's curators; nothing carries its release along. `move_skill` is for curators; you can do it too.
- **Personal skills** are private and released by the owner alone. Leavers' personal skills are blocked when a space is converted.
- **Distribution:** assistants load skills through the tools in every client. A client that supports the MCP skills extension (SEP-2640) can load them natively; a ZIP export of a released version is available at `/skills/export/<space>/<skill>` . Which clients use the extension has to be tested per client; the audit log shows which route a client takes (`via`).

Watch the audit log for `skill_publish` , `skill_approve` with `self_approved` , `skill_block` , `skill_move_*` and `skill_delete` (see below).

Audit log and monitoring

"*Show the last audit entries.*" (`admin_audit_log`). Filters: `user` (e-mail), `action` (exact or a prefix with `*` , for example `skill_*`), `space` , `document_id` , `skill` (skill name), `client` , `since` , `until` (RFC 3339 or `YYYY-MM-DD`), `limit` (default 50, at most 200).

What is recorded

Area	Actions
Knowledge	`store`, `update`, `archive`, `unarchive`, `move`, `delete`, review decisions (`approve`, `reject`, with `self` for self-approval by admins), `search` (hit count; query text only if the user's personal space was not searched), `handoff_save`
Skills	`skill_save`, `skill_approve` (risk class, digest, `self_approved`), `skill_reject`, `skill_publish`, `skill_block`, `skill_unblock`, `skill_archive`, `skill_unarchive`, `skill_delete`, `skill_tag`, `skill_move_out`, `skill_move_in`, reads `skill_get`, `skill_resource`, `skill_list`, `skill_export` (each with `via`: `tool`, `resource`, `rpc` or `export`)
Spaces and members	`admin_create_space`, `admin_update_space`, `admin_archive_space`, `admin_restore_space`, `space_add_member`, `space_remove_member`
Users and logins	`login`, `login_denied`, `account_linked`, `admin_offboard_user`, `admin_reactivate_user`, `admin_transfer_personal_space`, `revoke_my_sessions`, `admin_revoke_sessions`, `admin_allowlist_add`, `admin_allowlist_remove`, `admin_group_*`, `purge_personal`
Refusals	`denied` (tool and reason; role refusals, approval required, rate limits)
Instance	`admin_context_set`, `admin_context_clear`, `admin_quota_set`, `license_install`, `license_remove` (scope, target, old and new values)

Each entry has time, user, client label (`claude`, `chatgpt`, `claude-code`, `mcp-skills` for the `skills/*` methods, `api`, `system` for server-side events such as logins), action, space and details.

Useful questions

- *Everything that happened to one skill:* `skill=code-review`.
- *Who released what, and who approved their own:* `action=skill_publish`, `action=skill_approve`.
- *Who reads which skill and by which route:* `action=skill_get` with `client=chatgpt`; look at `via`.
- *What was moved or deleted:* `action=skill_move*`, `action=delete`, `action=skill_delete`.
- *Failed logins:* `action=login_denied`.
- *Changes to members and spaces:* `action=space_add_member`, `action=admin_*`.
- *What happened in a space last week:* `space=Support since=2026-11-01` (searches are not tied to a space).

Limits to know

- **Personal spaces** appear only as "personal space" with time and action, never with names, titles or contents.
- **Denied attempts** are logged as `denied` with the tool and the reason (`role does not allow this`, `approval required`, `own version cannot be approved`, `no permission for this tool`, `write rate limit`, `read rate limit`), no arguments, at most once a minute per user, tool and reason. A missing item (`not found`) is not logged, because it may simply not exist. There are **no alerts**; the log is only read by you. Plan a regular review (see Routines).

- The log lives in the same database as the content. Anyone with database administration rights could alter it. Protect the database accordingly; an off-box copy and tamper protection are on the roadmap.
- For request-level detail set `instance.log_level: debug`: the server then logs one line per MCP request (method, tool, resource, client) to its log.

Operating the system

Health

"*Show the Atrium health.*" (`admin_health`) reports the database, the embedding service, background jobs and tasks, and the licence with its notice. The TLS certificate is managed by the proxy and not checked by Atrium, so monitor its expiry there. `/healthz` and `/readyz` are for monitoring tools; `/readyz` answers only `ok` or `not ready`, and the reason is in the log and in `admin_health`.

Support information

"*Collect the Atrium support information.*" (`admin_support_bundle`) gathers what support needs in one answer: version, release line and build, licence state and notice, health, the versions of Go, Postgres and pgvector, the migration, the database size, counts of users, spaces, entries, skills and audit entries, and the settings of `config.yaml`. It contains **no content, no names of people or spaces and no secrets**: settings appear by a fixed list, without passwords, keys, client IDs, addresses of other systems, allowed domains or the organisation's texts. Read it before you pass it on. Each call is audited as `support_bundle`.

Backups

`scripts/backup.sh` writes a compressed `pg_dump` and removes old dumps; schedule it daily. The dump contains personal spaces, so protect it like the database. Restore into an empty database and test it once. Embeddings are in the dump; changing the embedding model needs a reindex.

Updates

- **Releases** are tagged `vX.Y.Z`; use a fixed version in your compose file and update on purpose. **Canary** images (`canary`) follow the development branch: for a test instance only, migrations only go forward, so take a backup first. See Releases.
- Read the **Upgrade notes** in the changelog before every update: the server refuses to start with some old settings, and new limits may apply.
- Configuration changes need a restart; the allowlist, context, groups, spaces and members change at runtime.

The licence

Without a licence the instance is the free **Community** edition. A licence is a signed line of text that switches limits and features on, is bound to the host of `instance.base_url`, and is checked **offline**: nothing calls anybody. Ask *"Show the Atrium licence."* (`admin_license action=get`) to see the state (`none`, `valid`, `not covered`, `expired`, `wrong domain`, `invalid`), what the licence covers (features, limits, the maintenance date), the release line of the running build and how many accounts are active against the user limit in force, with a warning when the limit is reached; *"Install this licence: atrium-license-v1…."* installs one without a restart (after your confirmation), *"Remove the installed licence."* deletes it after telling you how many accounts are active and what the Community limit means for them. Every change is audited (`license_install`, `license_remove`), the licence text itself is not.

- **Maintenance, not an expiry.** A licence carries a date, `maintenance_until`. A build is covered, for ever, when the release line it comes from was released on or before that date. You can run the version you have indefinitely; patch releases of your line are always covered; a **newer line** needs a licence with a later date, which you install once, in the running instance, usually once a year. After an update that says `not covered`, either go back to the covered version or renew the maintenance.
- **Notice ahead of time.** From 60 days before the end of maintenance, and before the `expires` date of a trial or subscription licence, the licence report, `admin_health` and the server log at start carry a notice with the date. After the end of maintenance the notice stays as a reminder that newer lines need a renewed licence; the running build is not affected.
- **Nothing is ever blocked.** In every state other than `valid` the instance runs as Community, and the content stays readable and exportable. The one hard limit built so far is the **number of users**: a new account beyond the limit is refused with a message, existing accounts and the administrators in `bootstrap_admins` are never affected. Without a licence that sets it, the Community limit is **8 active accounts** (`license.community_max_users` in `config.yaml`, 0 = unlimited). It counts registered accounts that are active at the same time, not connected assistants; locked accounts (after `admin_offboard_user`) do not count, so offboarding frees the seat, and reactivating needs room again. Administrators from `bootstrap_admins` count but are never refused.
- **Removing a licence by mistake locks nobody out.** If more accounts are active than the limit then in force (after a removal, an expiry, or an update beyond the maintenance), everybody keeps working; the licence report shows `over_limit` and a warning, and only new accounts and reactivations wait until there is room or a covering licence is installed again.
- A licence from `license.file` works too; one you install at run time takes precedence.

See Licences.

Separate database accounts

Out of the box the server connects to the database as its superuser. That is simple, but a flaw in the server (or anybody holding its connection string) could switch off the triggers that freeze released skills, rewrite the audit log or change the schema. The server logs a warning at start while it runs as a superuser. For production, set up two accounts once, with `scripts/db-roles.sql`:

- `atrium_app` (the server): reads and writes the data, can only **append** to the audit log, cannot change the schema, create or drop tables or switch triggers off.
- `atrium_migrate`: owns the schema and runs the migrations at start.

Then set `database.url` to the first and `database.migration_url` to the second in `config.yaml`. The steps are in Deployment. An update that adds tables works without further steps.

Limits that protect the system

Per user: 120 reads and 20 writes a minute. Per address: 20 client registrations, 60 login steps and 300 token requests a minute. These rates are set in `config.yaml`. The storage limits are quotas and change at runtime. The limits per address rely on `trusted_proxies` being right.

Embedding and search

Search uses an embedding service you choose (your own TEI server, OpenAI-compatible, Amazon Bedrock). Memory needs, flags for small servers and data protection notes are in Embedding. Changing the model means reindexing.

Security responsibilities and incidents

Ongoing.

- Keep the instance-admin list short and use MFA at the identity provider.
- Review **members and roles** of each space every quarter (ask space admins), and the **allowlist**.
- Restrict allowed domains to what you need; prefer single addresses for guests.
- Check `allowed_redirect_uris` only lists the assistants you use.
- Update regularly (see changelog security notes). Keep the host, proxy and database current.
- Treat the **database and backups** as sensitive: they contain everything, including personal spaces.
- Do not set `dev_mode` outside a developer's machine.

Incident playbook.

Situation	Steps
An account may be compromised	`admin_revoke_sessions` for the user; if confirmed, `admin_offboard_user` to lock it; review `admin_audit_log user=...` for what it did; reset access at the identity provider
A harmful or wrong skill	Block the version (curator or you), check who loaded it (`skill=..., action=skill_get`), tell the users, have a fixed version reviewed. Delete only if it must disappear
Sensitive data in an entry	Archive at once to stop it being served, then delete it for good (that works on archived entries); reads of knowledge entries are not logged, so the audit log shows who changed it and which searches ran, not who read it; tell the data protection officer
Credentials leaked in an entry	Treat them as compromised, rotate them first, then delete the entry

Situation	Steps
Unwanted login attempts	`login_denied` entries; tighten the allowlist; check provider settings
A curator or author misbehaves	Remove their role (space admin or you); review their recent changes in the audit log
Instance admin account needs to change	Edit `auth.bootstrap_admins` and restart

Known limits (be honest with your organisation). No notifications or alerts yet; the audit log is not tamper-proof against database administrators; unless you set up the separate database accounts the application connects to the database with a powerful account; and embedding services may process content outside your network if you choose a cloud backend. These points are on the roadmap.

Routines

Daily or on demand

- Check `admin_health` after incidents or updates.
- Answer requests from space admins (spaces, approval, groups, class C skills).

Weekly

- Skim `admin_audit_log` for `denied` (who keeps trying things their role does not allow), `login_denied`, `skill_publish`, `skill_approve` with `self_approved`, `skill_block`, `skill_delete`, `delete`, `admin_*`.
- Check that backups ran.

Monthly

- List users (`admin_list_users`) and compare with HR: lock leavers (offboarding).
- Read the changelog and plan updates.

Quarterly

- Access review: members, roles, groups and the allowlist, space by space with the space admins.
- Skill review: which skills exist, which are used (audit reads), which are stale; archive what is unused.
- Test a **restore** of the backup into an empty database.
- Review the context and space descriptions for accuracy.
- Look at the usage of spaces that fill up (`admin_quotas`) and adjust the limits before people hit them.

New space. Request from a team lead, check purpose and audience, decide approval and default role, name two curators and an admin, create it, announce it.

New employee. Allowlist or domain covers them; they log in once; the space admin adds them.

Leaver. HR notice, offboarding procedure, decision on the personal space, group and space clean-up, documentation.

Update. Read the upgrade notes, backup, update on a test instance if you have one, then production, check `admin_health` and a test login.

Troubleshooting

Problem	What to try
Nobody can log in	Check the identity provider settings and the allowlist; `login_denied` in the audit log names the reason. Check `/readyz` and the server log
A user cannot be added to a space	They must have logged in once
Assistants do not know the new context	Only new connections get it. Start a new chat
`/readyz` says `not ready`	Look at `admin_health` and the server log (database or embedding service)
The server does not start after an update	Read the log: the upgrade notes list settings that are now refused (for example `dev_mode` on a non-loopback address, a wrong redirect pattern)
Search finds nothing new	Indexing is asynchronous; check the embedding service and the health of background tasks
Limits trigger for many users at once	Check `trusted_proxies`: if wrong, all users share one address
A class C skill is stuck	Only an instance admin can publish it: review and publish, or ask the curators to fix it
You cannot read a personal space	By design; use the offboarding transfer for leavers

FAQ

Can I read an employee's personal space? No. Not even for investigations. For leavers, transfer it after locking the account.

Can I see who searched for what? Searches are logged, but with the query text only when the search did not include the user's personal space (for example when the assistant restricted it to team spaces); otherwise only the number of hits is kept. Search events are not tied to a space, and reads of single entries are not logged.

How do I make someone an instance admin? Add the address to `auth.bootstrap_admins` and restart.

Can I turn approval on for an existing space? Yes (`admin_update_space`). Existing entries keep their status.

What happens to a space I archive? It disappears for everybody except instance admins; content is kept and can be restored.

Can I change someone's role in a space? Yes, as admin of every team space (`space_add_member`). Prefer asking the space admin.

How long are logins valid? Access token one hour, refresh token 30 days, absolute 90 days.

Where do I find the technical documents? In the `docs/` folder of the repository, linked from this guide.

Do users need to be told about the audit log? Yes. Say what is recorded (actions, not contents of personal spaces) and who can read it.

Can I export the audit log? Not as a file yet. Query it with filters (at most 200 entries per call, narrow by time or action).

Is there a notification when drafts wait for review? Not yet. Agree turnaround times with curators.

I removed the licence and have more users than Community allows. Are they locked out? No. Every active account keeps working; only new accounts and reactivations are refused until there is room. "*Show the Atrium licence.*" shows the active accounts against the limit; installing the licence again lifts it at once.

What do I send when I ask for support? The answer of "*Collect the Atrium support information.*" (`admin_support_bundle`), the time of the problem and what you asked. It holds no content, names or secrets; read it before you send it.

How do I know that maintenance is about to end? The licence report and `admin_health` show a notice from 60 days before, and the server log warns at start. Renew in time if you want the next release line.